UNIVERSITY OF CHITTAGONG

Department of Computer Science and Engineering

Session: 2021-2022
6th Semester
Deliverable: 01
Course Title: Web Engineering Lab
Course Code No.: CSE 616
Program: BSc. (Engineering)

Submitted to:
Rokan Uddin Faruqui
Professor
Department of Computer Science and Engineering
University of Chittagong

Submitted by:

Name	Id
Adnan Uddin	22701011
Kamrul Arafin	22701016
Meheer Ali Khan	22701034

Date of Submission: 17 September 2025

1 Project Overview 2
1.1 Project Objectives 2
Functional and non-functional requirements 2
1. Functional Requirements 2
1.1 User Authentication & Authorization 2
1.2 Patient Management 3
1.3 Doctor Dashboard 3
1.4 Administrative Features 3
1.5 Additional Features 3
2. Non-Functional Requirements 4
2.1 Security Requirements 4
2.2 Performance Requirements 4
2.3 Reliability & Availability Requirements 4
2.4 Scalability 4
2.5 Usability 4
2.6 Maintainability Requirements 5
2.7 Deployment Requirements 5
User Stories with Acceptance Criteria 5
1. Authentication & Authorization 5
2. Patient Management 5
3. Doctor Dashboard 6
4. Administrative Features 6
5. Additional Features 6
Technology stack justification 7
1. Frontend: Next.js (React-based Framework) 7
2. Backend: Node.js with Express.js 8
3. Database: PostgreSQL (SQL Database) 8
4. ORM: Sequelize (for Node.js & PostgreSQL) 8
5. Deployment & DevOps 9
6. Project Management & Collaboration Tools 9
Team roles and responsibilities 9
1. Scrum Master 9
2. Frontend Developer 9
3. Backend Developer 10
4. DevOps Engineer 10
5. QA Engineer 10
Project timeline and milestones 10
Milestone Summary 11
Risk assessment and mitigation strategies 11

Healthcare Web Application Development – Capstone Project

Course Code: CSE 616 – Lab on Web Engineering
Department of CSE, University of Chittagong
Date: September 17, 2025

1 Project Overview

This capstone project requires teams to develop a comprehensive healthcare web application following modern web engineering practices and agile software development methodologies. The project spans 4 months with 5 major deliverables scheduled at 2-week intervals.

1.1 Project Objectives

• Design and implement a full-stack healthcare web application
• Apply agile software engineering principles throughout development
• Demonstrate proficiency in modern web technologies
• Implement proper deployment and optimization strategies
• Document the entire development process

Functional and non-functional requirements

1. Functional Requirements

These are features the system **must perform** to meet user needs.

1.1 User Authentication & Authorization

- Multi-role system: Patient, Doctor, Admin
- Secure login/logout
- Password reset & account recovery
- Role-Based Access Control (RBAC)

1.2 Patient Management

- Patient registration and profile management

- Medical history tracking (diagnoses, allergies, medications)
- Appointment scheduling system (view, book, cancel)

- Upload/view medical documents and lab results

1.3 Doctor Dashboard

- View and manage appointments
- Access patient medical records
- Write, manage, and track prescriptions
- Optionally, conduct telemedicine/video consultations

1.4 Administrative Features

- User management (add/remove/modify users)
- Manage hospital/clinic details (departments, doctors)
- Generate analytics and reports on appointments, patient visits, etc.

1.5 Additional Features

Medical Imaging Upload & Viewing

- Patients can upload medical images (X-ray, MRI, CT scans)
- Doctors can view and annotate images securely

Medication Tracking & Reminders

- Patients receive automated reminders for medications
- Track dosage and adherence over time

Health Monitoring Dashboard

- Track BMI, vital signs, and other health trends
- Visualize historical data in charts and reports

Lab Test Integration & Results Viewing

- Patients can view lab test results in their profile
- Doctors can update and annotate test results

2. Non-Functional Requirements

These define **how well** the system performs and behaves.

2.1 Security Requirements

- HTTPS for all communication
- Proper input validation and sanitization
- JWT tokens for authentication
- Password hashing with salt
- Role-based access control (RBAC)
- Patient data encrypted in transit and at rest
- Protection against common vulnerabilities (SQL Injection, XSS)

2.2 Performance Requirements

- API response time < 500ms (target)
- Page load < 3 seconds (target)
- Database queries optimized
- Caching implemented (Redis or Memcached)

2.3 Reliability & Availability Requirements

- Database backup and recovery plan
- Error handling and logging
- System uptime ≥ 99% (as practical in deployment)

2.4 Scalability

- Modular architecture for adding new features
- Support for 100+ concurrent users

2.5 Usability

- Responsive design (desktop, tablet, mobile)
- Intuitive navigation and clear UI
- Accessibility considerations (ARIA labels, keyboard navigation)

2.6 Maintainability Requirements

- Well-documented code
- Consistent coding standards
- Modular components for easier updates

2.7 Deployment Requirements

- Deployment via Docker containers for platform independence
- Cloud-ready (Heroku, AWS Free Tier, or GCP)

- Environment configuration management
- Database backup and recovery plan
- Monitoring and logging setup

3. User Stories with Acceptance Criteria

3.1 Authentication & Authorization

1. **As a patient**, I want to register and log in so that I can access my health information securely.
    - **Acceptance Criteria:**
        - Patient can create an account and log in.
        - Passwords are hashed and stored securely.
        - Login fails for invalid credentials.
2. **As a doctor**, I want to log in with my role so that I can access doctor-specific features.
    - **Acceptance Criteria:**
        - Doctor sees only authorized sections (dashboard, patient records).
        - Role-based access control prevents unauthorized access.
3. **As an admin**, I want to manage users and roles so that system access is controlled.
    - **Acceptance Criteria:**
        - Admin can create, update, or delete patient and doctor accounts.
        - Admin can assign or modify user roles.

3.2 Patient Management

4. **As a patient**, I want to update my profile so that my medical information is accurate.
    - **Acceptance Criteria:**
        - Patient can edit profile details (contact info, medical history).
        - Changes are saved and visible on next login.
5. **As a patient**, I want to schedule appointments so that I can see doctors conveniently.
    - **Acceptance Criteria:**
        - Patient can book, cancel, and view appointments.
        - Appointment confirmation is displayed.

3.3 Doctor Dashboard

6. **As a doctor**, I want to view patient records so that I can provide accurate treatment.
    - **Acceptance Criteria:**
        - Doctor can search and access assigned patient records.
        - Patient medical history and lab results are visible securely.
7. **As a doctor**, I want to manage prescriptions so that treatment is tracked properly.
    - **Acceptance Criteria:**
        - Doctor can create, update, or delete prescriptions.
        - Prescriptions are linked to patient profiles.

3.4 Administrative Features

8. **As an admin**, I want to view system analytics so that I can monitor hospital operations.
    - **Acceptance Criteria:**
        - Admin can view reports on user activity, appointments, and clinic usage.
        - Analytics are displayed in charts and tables.

3.5 Additional Features

9. **As a patient**, I want to upload and view medical images so that my doctors can access them.
    - **Acceptance Criteria:**
        - Patient can upload images (X-ray, MRI, CT) securely.
        - Doctor can view and annotate images.
10. **As a patient**, I want to track my medications and get reminders so that I take them on time.
    - **Acceptance Criteria:**
        - Patient can enter medication schedule.
        - System sends reminders at scheduled times.
11. **As a patient**, I want a health monitoring dashboard so that I can track BMI, vitals, and trends.
    - **Acceptance Criteria:**
        - Dashboard displays historical data in charts.
        - Alerts are generated for abnormal trends (optional).

12. **As a patient**, I want to view lab test results so that I can track my health progress.

    - **Acceptance Criteria:**

        - Lab results are securely uploaded by doctors or lab staff.
        - Patient can view results and download reports.

4. Technology stack justification

4.1 Frontend: Next.js (React-based Framework)

- **Reason for Selection:**

    - Supports **Server-Side Rendering (SSR)** and **Static Site Generation (SSG)** for faster page loads and better SEO.
    - Component-based architecture makes UI modular, maintainable, and reusable.
    - Large ecosystem with ready-to-use libraries (React Hooks, Material UI, Tailwind CSS).
    - Easier integration with **backend APIs** via REST or GraphQL.

- **How it fits project requirements:**

    - Patient dashboard, doctor dashboard, and health monitoring features require **dynamic, interactive interfaces**.
    - Mobile-responsive UI is straightforward to implement.

4.2 Backend: Node.js with Express.js

- **Reason for Selection:**

    - **Non-blocking, event-driven architecture** ideal for real-time features like appointment notifications, medication reminders, and video consultations.
    - Rich ecosystem of packages via **npm**, including libraries for JWT authentication, encryption, and REST API development.
    - Easy to maintain and extend as team members already use JavaScript in frontend.

- **How it fits project requirements:**

    - Handles **RESTful API endpoints**, user authentication, medical record management, and admin analytics efficiently.
    - Scales well for concurrent users.

4.3 Database: PostgreSQL (SQL Database)

- **Reason for Selection:**

    - **Structured relational database** perfect for managing complex relationships (patients ↔ doctors ↔ appointments ↔ prescriptions).
    - ACID compliance ensures data consistency for sensitive medical information.
    - Strong community support and ORMs available (Sequelize) for rapid development.

- **How it fits project requirements:**
    - Enables **secure storage** of medical history, lab results, prescriptions, and imaging metadata.
    - Supports **query optimization** for analytics and dashboards.

4.4 ORM: Sequelize (for Node.js & PostgreSQL)

- **Reason for Selection:**

    - Simplifies database operations with object-relational mapping.
    - Supports migrations, model associations, and validations.

- **How it fits project requirements:**

    - Ensures **clean code and maintainability** for database operations.
    - Helps implement **secure and validated queries**, preventing SQL injection.

4.5 Deployment & DevOps

- **Docker:** Containerization ensures consistent environment across development, testing, and production.
- **Cloud Hosting (Vercel/Heroku/AWS Free Tier):** Simplifies deployment and scaling.
- **Redis (Optional Caching):** Improves API response time for frequent queries (appointments, health dashboard data).
- **How it fits project requirements:**

    - Enables **fast deployment, monitoring, and scaling**.
    - Supports **high availability and performance optimization**.

4.6 Project Management & Collaboration Tools

- **Trello:** Agile sprint planning, task tracking, backlog management.
- **Slack:** Team communication, daily check-ins, integration with GitHub & Trello.
- **GitHub:** Version control with branching strategy for collaborative development.
- **How it fits project requirements:**

    - Ensures **efficient collaboration**, documentation, and process transparency.
    - Supports **Agile methodology** adherence.

5. Team roles and responsibilities

5.1 Scrum Master

- **Responsibilities:**
    - Coordinate all agile processes (sprint planning, daily standups, retrospectives).
    - Maintain and update **Trello board** with backlog items, sprints, and priorities.
    - Ensure the team adheres to project timeline and deadlines.
    - Facilitate communication between team members and instructor/mentors.
    - Resolve conflicts or blockers affecting development progress.

5.2 Frontend Developer

- **Responsibilities:**
    - Develop the user interface using **Next.js**.
    - Implement responsive, interactive dashboards for patients, doctors, and admins.
    - Integrate **frontend with backend APIs** for real-time features (appointments, health monitoring, lab results).
    - Ensure cross-browser compatibility and accessibility.
    - Implement frontend testing (unit and integration tests) and optimize performance.

5.3 Backend Developer

- **Responsibilities:**
    - Implement server-side logic using **Node.js + Express.js**.
    - Develop **RESTful APIs** for authentication, patient management, doctor dashboard, and admin analytics.
    - Manage database models, migrations, and queries using **Sequelize ORM with PostgreSQL**.
    - Ensure data security, role-based access control, and encryption for sensitive medical information.
    - Write unit and integration tests for backend functionality.

5.4 DevOps Engineer

- **Responsibilities:**
    - Set up **Docker containerization** for consistent development and production environments.
    - Deploy the application to cloud platforms (**Vercel, Heroku, or AWS Free Tier**).
    - Implement caching strategies (**Redis/Memcached**) and monitor performance metrics.
    - Set up monitoring, logging, and alerts for system health.
    - Ensure secure environment configuration and database backup/recovery plan.

5.5 QA Engineer

- **Responsibilities:**
    - Plan and execute **manual and automated tests** for both frontend and backend.
    - Ensure all functional requirements and user stories are tested.
    - Track, report, and follow up on bugs in GitHub Issues.
    - Validate **performance, usability, and security** requirements before deployment.

6. Project timeline and milestones

Week	Deliverable	Due Date	Components
2	D1 – Requirements	September 17, 2025	Requirements document, tech stack selection, project setup
4	D2 – Design	October 5, 2025	System architecture, UI/UX mockups, database schema
6	D3 – Backend MVP	October 19, 2025	API endpoints, authentication, database integration
8	D4 – Frontend Integration	November 2, 2025	Complete UI implementation, frontend-backend integration
10-12	D5 – Deployment & Final	November 16 and 30, 2025	Production deployment, optimization, final documentation

Milestone Summary

1. **D1 – Requirements & Tech Stack:** Foundation for all development.
2. **D2 – Design:** Blueprint for UI, backend, database, and security.
3. **D3 – Backend MVP:** Core API endpoints, authentication, database models.
4. **D4 – Frontend Integration:** Full UI, API integration, responsive design.
5. **D5 – Deployment & Final Report:** Live application, optimized, secure, and documented.

7. Risk assessment and mitigation strategies

Risk	Description	Impact	Likelihood	Mitigation Strategy
Security Breach	Unauthorized access to sensitive patient data	High	Medium	- Implement HTTPS for all communications - Use JWT for authentication and RBAC - Encrypt patient data at rest and in transit - Regularly update dependencies and perform vulnerability scans
Backend Performance Issues	Slow API responses or server crashes under load	High	Medium	- Optimize database queries and use indexing - Implement caching using Redis/Memcached - Conduct load testing and monitor performance
Data Loss	Accidental deletion or corruption of database	High	Low	- Implement regular database backups - Use transactional operations for critical data - Establish recovery plan and test it periodically
Deadline Pressure / Team Delays	Tasks not completed on time affecting project timeline	High	Medium	- Use Agile methodology with clear sprints and story points - Weekly sprint planning and daily standups - Prioritize tasks and monitor progress on Trello
Cross-browser or Device Issues	UI not responsive or features fail on certain browsers/devices	Medium	Medium	- Conduct frontend testing across browsers and devices - Use responsive design frameworks (Tailwind CSS/Bootstrap) - Fix compatibility issues during development cycles
Integration Issues	Frontend and backend APIs fail to integrate correctly	High	Medium	- Define API contracts clearly in design phase - Implement automated integration testing - Maintain version control and staging environment
Knowledge Gaps	Team members unfamiliar with certain tools/technologies	Medium	Medium	- Assign roles based on expertise - Conduct short knowledge-sharing sessions - Refer to official documentation and tutorials